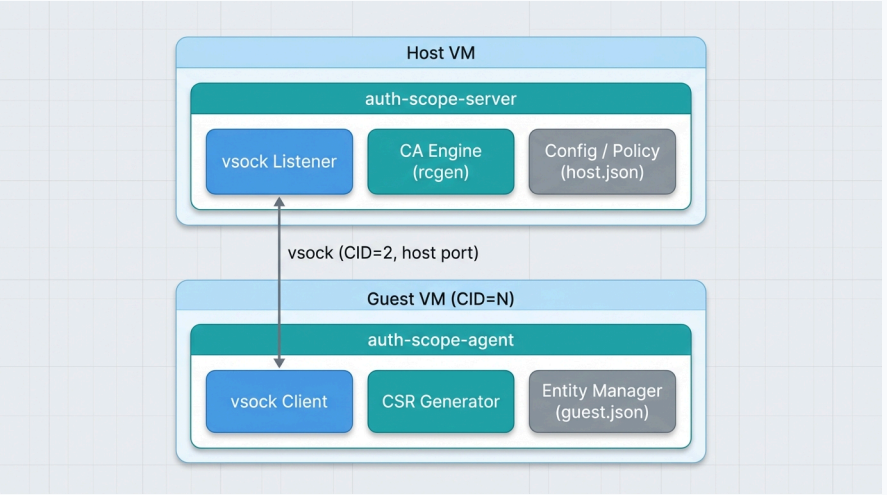
Auth-Scope vs. SPIFFE/SPIRE

Hypervisor-Attested Workload Identity in Multi-VM Environments

A comparative study of vsock-based and network-based PKI bootstrapping

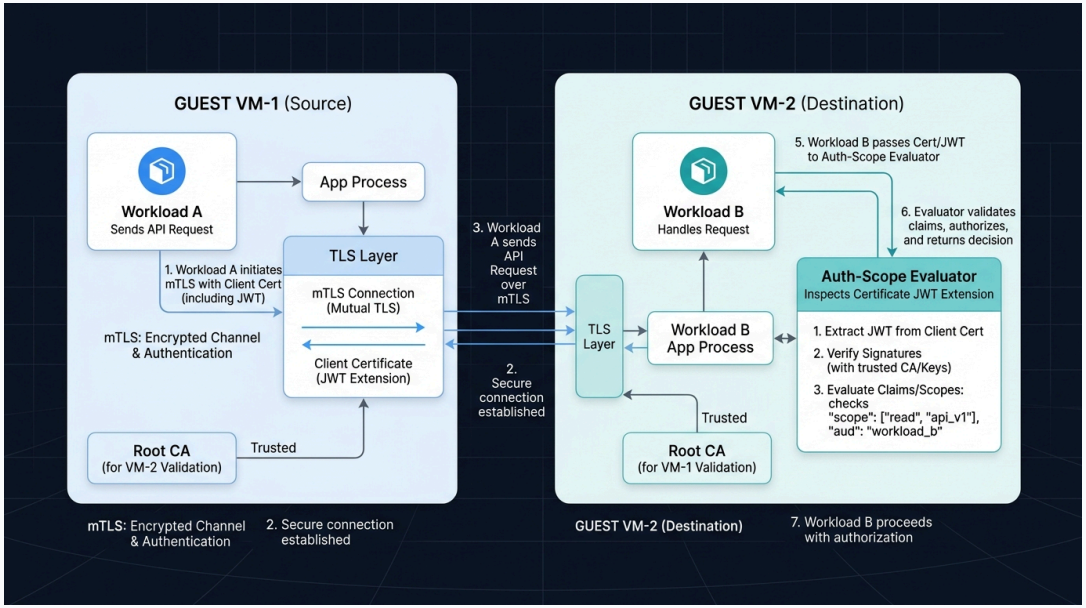
1. Auth-Scope Overview & Architecture

- **Vsock-Based Handshake**:
  - Guest agents communicate with the host CA over virtual sockets (`vsock`).
  - Host server uses kernel-enforced `peer_cid` socket metadata to verify the guest VM's identity.
  - **No network-based trust bootstrapping needed.**
- **Embedded Capabilities**:
  - Dynamic X.509 certs contain capability claims (allowed paths, RPC modules) encoded as a JWT extension.
- **Host-Locked Port Security**:
  - Guest agent runs as a persistent service, locking port `901` to prevent unprivileged hijacking.



2. Peer-to-Peer Attestation in Multi-VMs

- **Workload Identity Provisioning**:
  - Local guest agents generate CSRs and fetch credentials from the Host CA.
- **mTLS Peer-to-Peer Verification**:
  - Workload A on VM-1 connects to Workload B on VM-2 over standard mTLS.
- **Offline Local Evaluation**:
  - Workload B uses SDKs to verify Workload A's certificate capabilities offline.
  - **Zero network dependency** on the CA server during traffic flow.



3. Auth-Scope vs. SPIFFE/SPIRE

Feature	SPIFFE / SPIRE	Auth-Scope
Trust Channel	TCP/IP network (gRPC over mTLS)	Hypervisor Virtual Sockets (`vsock`)
VM Bootstrapping	Complex (Requires join tokens, cloud APIs, OIDC, or TPM/VTPM)	**Implicit & Instant** via host-kernel `peer_cid` socket metadata
Network Dependency	High (Requires IP routing, DNS, and central spire-server access)	**Zero** (Agent-to-Host vsock requires no TCP/IP stack)
Authorization	Separate (SPIRE handles identity; OPA/Authz done externally)	**Built-in** (Granular capability claims are embedded in the certificate JWT)
Footprint	Heavy (Java/Go daemon stack, database, complex setup)	**Ultra-lightweight** (< 3MB Rust binaries, zero runtime databases)